MedTools — Project Handoff & Dev Guide

For Ben's brother — read this before touching anything

Section 1: What Is This?

MedTools is a web app built for medical students. It runs on Google Cloud and is live at two URLs:

Main URL (use this): https://medtools-77dfb.web.app

Backup (if main is blocked by your network): https://medtools-509459643164.us-central1.run.app

The whole app is **ONE file: app.py**. Everything — the server logic, HTML, CSS, and JavaScript — lives in that single file. It is a Python Flask app.

Section 2: The Four Tools

1. Anki Card Generator

Upload a lecture PDF → Claude reads it → spits out Anki flashcards → download as .apkg file and import directly into Anki.

2. Practice Tests

Upload lecture PDFs → Claude generates 15 NBME board-style multiple choice questions → interactive quiz with scoring → Download PDF button opens a print-ready version. Uses Claude Haiku (fast model) for speed.

3. UWorld Review

Paste your missed UWorld question IDs → the browser talks directly to Anki running on your computer via AnkiConnect → pulls the matching AnKing deck cards → Claude analyzes what you missed → gives a breakdown + 10 drill questions. Requires AnkiConnect add-on installed in Anki, and the site domain added to AnkiConnect's CORS whitelist.

4. Jeremy Mode

An AI tutor chat. Upload a PDF or type a question. Jeremy explains concepts, builds mnemonics, quizzes you. Responses stream in real-time like ChatGPT.

Section 3: Tech Stack

Backend: Python / Flask

AI: Anthropic Claude API (claude-sonnet-4-6 for most things, claude-haiku-4-5 for practice tests)

Hosting: Google Cloud Run (the server) + Firebase Hosting (the domain)

Database: Firebase Firestore (user library / saved tests)

Auth: Firebase Google Sign-In

Card generation: genanki Python library

All frontend: inline HTML/CSS/JS inside app.py (no separate frontend framework)

Section 4: How to Set Up Locally

Step 1 — Clone the repo

Open a terminal (Windows: press Win+R, type cmd, hit Enter) and run:

```
git clone https://github.com/BenSeamons/Anki.git
cd Anki
```

Step 2 — Create a Python virtual environment

```
python -m venv .venv
.venv\Scripts\activate
```

Step 3 — Install dependencies

```
pip install -r requirements.txt
```

Step 4 — Add the secret API key

Create a file called **.env** in the Anki folder (no other name, just .env) and paste:

```
ANTHROPIC_API_KEY=sk-ant-api03-[get this from Ben privately]
```

Step 5 — Run it

```
python app.py
```

Step 6 — Open in browser

Go to: http://localhost:5000

Section 5: How GitHub Works (for vibe coders)

Think of GitHub as Google Drive for code. Every change you make needs to be saved in three steps:

Step 1 — Stage your changes (tell git what files changed):

```
git add .
```

Step 2 — Commit (write a note about what you changed):

```
git commit -m "describe what you did here"
```

Step 3 — Push (upload to GitHub so Ben can see it):

```
git push origin main
```

To get Ben's latest changes onto your machine:

```
git pull origin main
```

Golden rule: always **git pull** before you start working, always **git push** when you are done.

Section 6: How to Deploy (Make Changes Go Live)

After pushing to GitHub, the site does NOT auto-update. To push changes live:

Step 1 — Make sure gcloud is installed and you are logged in:

```
gcloud auth login
```

Step 2 — Deploy to Cloud Run:

```
cd "C:\path\to\Anki"
gcloud run deploy medtools --source . --project medtools-77dfb --region us-central1 --quiet
```

Step 3 — Firebase Hosting (only if firebase.json changed — usually skip this):

```
firebase deploy --only hosting
```

Deployment takes about 3-4 minutes. **Note: Only Ben has gcloud/Firebase access set up right now. Coordinate with him for deploys, or ask him to add your Google account to the project.**

Section 7: Your First Task — The Progress/Development Tab

Ben wants you to build a new **Progress** tab on the site. Here are some ideas — pick what excites you, or invent your own:

Ideas to consider:

Study streak tracker — show how many days in a row the user has generated practice tests or used UWorld review

Performance dashboard — track scores across practice tests over time (the app already records correct/total per test)

Weak topic identifier — analyze which topics keep showing up in UWorld missed questions

Usage stats — how many cards generated, how many tests taken, how many questions drilled

Upcoming features roadmap — a public-facing what's-coming-next page

Where to add it:

1. The nav bar is in the **BASE_HTML** variable near the top of app.py — add a new link like the others (Anki Generator, Practice Tests, Jeremy Mode, UWorld Review).
2. Create a new route: **@app.route('/progress')** in app.py.
3. Build the page HTML inline, just like the other pages.

How to test:

```
python app.py
# go to http://localhost:5000/progress
```

When it looks good: git add . && git commit -m "add progress tab" && git push, then coordinate with Ben to deploy.

Section 8: Key Files

app.py — THE file. Everything lives here. ~2400 lines.

requirements.txt — Python packages (flask, anthropic, genanki, gunicorn, python-dotenv)

Dockerfile — how the app gets containerized for Cloud Run

firebase.json — tells Firebase Hosting to forward all traffic to Cloud Run

static/venmo-qr.png — Ben's Venmo QR for the support widget

.env — YOUR secret API key (never commit this to GitHub)

.gitignore — list of files that should never go to GitHub (.env is already blocked)

Section 9: Important Rules

1. **Never commit .env to GitHub** — it has the API key. .gitignore blocks it but be careful.
2. **The whole app is in app.py** — when Ben says add a feature, he means edit app.py.
3. **HTML/CSS/JS goes inside Python strings** in app.py — it looks weird but that's how it works.
4. **Test locally before deploying** — python app.py and check http://localhost:5000 first.
5. **Coordinate deploys with Ben** — only one person should deploy at a time.

Good luck. Ask Ben if anything is unclear. He's grumpy but he'll help.